

1 Lập trình Di truyền

Lập trình Di truyền do John Koza phát triển từ cuối thập niên 1980 và được hệ thống hóa trong công trình năm 1992 [8]. GP là một mở rộng trực tiếp của GA, tương ứng với dạng *mã hóa dựa trên cấu trúc* đã được giới thiệu trong Mục ???. Khác biệt cốt lõi so với GA dùng mã hóa số thực hay hoán vị là: thay vì tiến hóa một chuỗi giá trị có độ dài cố định trong khuôn dạng nghiệm đã cho trước, GP tiến hóa trực tiếp **cấu trúc** của một biểu thức hoặc chương trình. Nhờ đó, thuật toán có khả năng tự khám phá đồng thời cả hình dạng lẫn tham số của nghiệm – đây chính là lý do GP được xem là bước chuyển từ “tối ưu tham số” sang “tự động tổng hợp chương trình”. Một tính chất đáng chú ý của GP là nghiệm cuối cùng có dạng *biểu thức tường minh*, có thể đọc, phân tích và giải thích được bởi con người, khác biệt căn bản với các mô hình hộp đen như mạng nơ-ron.

Mã hóa dạng cây biểu thức

Mỗi cá thể trong GP là một cây biểu thức. Mỗi nút trong của cây là một toán tử được chọn từ một *tập toán tử* cho trước, chẳng hạn các phép toán số học $\{+, -, \times, \div\}$ hoặc hàm lượng giác $\{\sin, \cos\}$; mỗi nút lá là một biến đầu vào hoặc hằng số được chọn từ một *tập biến và hằng số* cho trước. Khi duyệt cây theo đúng cấu trúc phân cấp của nó, ta thu được một biểu thức toán học tường minh, có thể tính giá trị trên bất kỳ bộ dữ liệu đầu vào nào. Điều này cho phép toàn bộ quần thể các cây với hình dạng và nội dung khác nhau được đánh giá, so sánh một cách thống nhất như trong GA thông thường.

Việc thiết kế hai tập hợp này cần thỏa mãn hai tính chất cơ bản:

- **Tính khép kín** (closure): mọi toán tử phải chấp nhận bất kỳ giá trị nào do một toán tử khác hoặc một nút lá trả về, và luôn cho ra một kết quả hợp lệ. Ràng buộc này nảy sinh vì lai ghép và đột biến ghép các toán tử với nhau một cách ngẫu nhiên, không theo trật tự cố định nào. Chẳng hạn, nếu $\div$ xuất hiện tại một nút mà cây con ở vị trí mẫu số vừa tính ra 0 – một tình huống hoàn toàn có thể xảy ra – phép chia thông thường không xác định, khiến cả cây không tính được giá trị và cá thể đó trở nên vô nghĩa. Để tránh điều này, $\div$ thường được thay bằng *phép chia bảo vệ*: trả về 0 (hoặc một hằng số lớn) khi mẫu số bằng 0, thay vì để phép tính thất bại. Các hàm có miền xác định hẹp hơn tập số thực như $\log$ hay $\sqrt{\cdot}$ cũng cần phiên bản bảo vệ tương tự, chẳng hạn $\log(|x|)$ thay cho $\log(x)$, để luôn nhận được mọi giá trị đầu vào có thể xuất hiện trong cây.
- **Tính đủ** (sufficiency): hai tập hợp này phải chứa đủ các thành phần để biểu diễn được (ít nhất xấp xỉ) nghiệm của bài toán. Nếu dữ liệu thực sự mang quan

hệ lượng giác mà tập toán tử chỉ gồm bốn phép tính số học, GP sẽ phải xấp xỉ hàm đích bằng đa thức với độ chính xác thấp.

Độ sâu tối đa của cây cũng là một tham số thiết kế quan trọng: giới hạn quá chặt sẽ thu hẹp không gian khám phá, trong khi không giới hạn đủ mức dễ dẫn đến các cây có kích thước rất lớn, tốn kém khi đánh giá và khó diễn giải.

Khởi tạo quần thể. Khác với GA nơi quần thể ban đầu chỉ là các chuỗi ngẫu nhiên, việc sinh cây ngẫu nhiên trong GP có ba chiến lược kinh điển, tất cả đều giới hạn độ sâu tối đa $d_{\max}$:

- **Full:** mọi nhánh của cây đều có độ sâu đúng bằng $d_{\max}$, tạo ra các cây cân đối, dày đặc;
- **Grow:** mỗi nhánh có thể kết thúc ở độ sâu bất kỳ không vượt quá $d_{\max}$, tạo ra các cây không đồng nhất về hình dạng;
- **Ramped half-and-half:** kết hợp hai phương pháp trên theo tỷ lệ 1 : 1, đồng thời phân bố độ sâu tối đa của từng cây đều trên khoảng $[2, d_{\max}]$. Đây là chiến lược được khuyến nghị và sử dụng phổ biến nhất, vì nó tạo ra quần thể ban đầu đa dạng cả về kích thước lẫn hình dạng, giảm nguy cơ quần thể bị chi phối bởi một kiểu cấu trúc duy nhất ngay từ đầu.

Hàm độ thích nghi

Ứng dụng tiêu biểu nhất của GP là bài toán **hồi quy ký hiệu** (symbolic regression): tự động tìm một biểu thức toán học mô tả tốt mối quan hệ giữa tập dữ liệu quan sát (X, y) mà không giả định trước dạng hàm. Bài toán này khác biệt căn bản so với hồi quy tuyến tính hay hồi quy đa thức, nơi dạng hàm đã được ấn định sẵn và công việc còn lại chỉ là tối ưu các hệ số. Trong GP, độ thích nghi của một cây thường được xây dựng từ sai số dự đoán, tiêu biểu là sai số bình phương trung bình giữa giá trị cây tính được và giá trị y thực tế. Một số công trình còn dùng RMSE, MAE, hoặc hệ số xác định R^2 ; lựa chọn này ảnh hưởng trực tiếp đến độ nhạy của chọn lọc đối với các ngoại lai trong dữ liệu.

Một vấn đề đặc trưng của GP là hiện tượng **bloat**: kích thước cây có xu hướng tăng trưởng liên tục qua các thế hệ mà không mang lại cải thiện tương xứng về độ chính xác. Nguyên nhân được lý giải ở mức lý thuyết là các cây con trung tính (ví dụ đoạn $+0, \times 1$) tích lũy dần vì chúng không làm giảm độ thích nghi nhưng lại được bảo vệ khỏi đột biến có hại. Bloat làm tăng chi phí đánh giá, giảm khả năng diễn giải nghiệm, và có thể dẫn đến quá khớp. Để đối phó, người ta thường cộng thêm vào hàm thích nghi một **số hạng phạt** tỉ lệ với kích thước cây:

$$\text{fitness}(T) = \text{MSE}(T) + \lambda \cdot |T|, \quad (1.1)$$

trong đó $|T|$ là số nút của cây T và $\lambda > 0$ điều chỉnh mức độ ưu tiên sự gọn nhẹ so với độ chính xác. Các biện pháp bổ sung gồm giới hạn độ sâu/số nút tuyệt đối, loại bỏ cá thể vi phạm ngay sau khi sinh, và các toán tử đột biến có chủ đích rút gọn cây.

Toán tử tiến hóa

Các toán tử lai ghép và đột biến trong GP thao tác trực tiếp trên cấu trúc cây thay vì trên chuỗi có độ dài cố định:

- **Lai ghép trao đổi cây con:** chọn ngẫu nhiên một nút trên mỗi cây cha mẹ, sau đó hoán đổi hai cây con bắt rễ tại các nút đó cho nhau để tạo ra hai cây con mới. Các biến thể gồm lai ghép bất đối xứng (cây con thứ nhất lấy từ nút trong, cây con thứ hai từ nút lá) và lai ghép tự thân (cắt và dán trên cùng một cây), nhằm cân bằng giữa khám phá và khai thác;
- **Đột biến thay thế cây con:** chọn ngẫu nhiên một nút trên cây, rồi thay toàn bộ cây con bắt rễ tại nút đó bằng một cây con mới được sinh ngẫu nhiên;
- **Đột biến điểm** (point mutation): thay một nút hàm bằng một hàm khác cùng số ngôi, hoặc thay một nút lá bằng biến/hằng số khác, giữ nguyên cấu trúc cây;
- **Đột biến rút gọn** (hoist mutation): thay toàn bộ cây bằng một cây con ngẫu nhiên bên trong nó, qua đó thu nhỏ cây một cách có kiểm soát – biến thể này thường được dùng kết hợp với số hạng phạt (1.1) để chống bloated.

Xác suất áp dụng của các toán tử cũng là tham số cần hiệu chỉnh: tỷ lệ dành cho lai ghép, đột biến và sao chép trực tiếp (reproduction) cộng lại bằng một, trong đó tỷ lệ lai ghép thường được đặt cao hơn hẳn tỷ lệ đột biến; giá trị cụ thể tùy vào bài toán và được xác định bằng thực nghiệm.

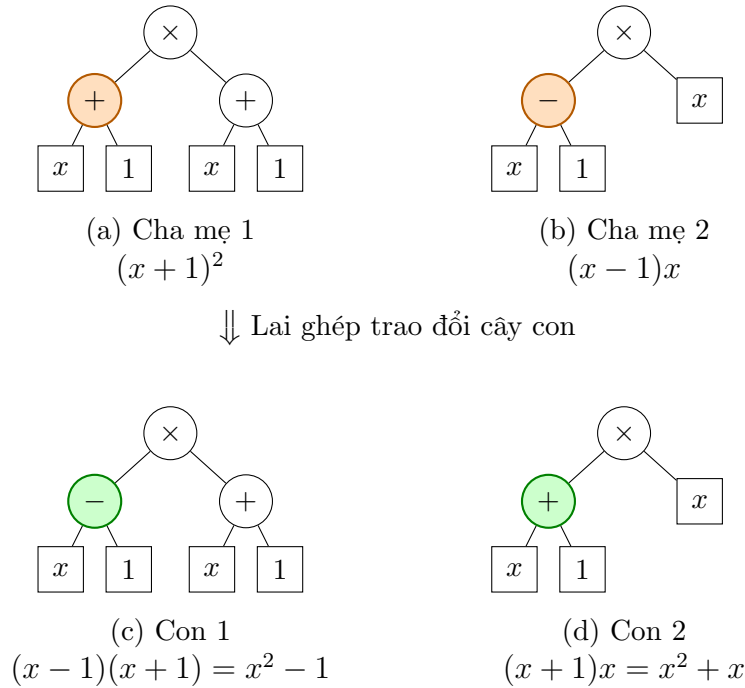
Các thành phần còn lại của chu trình tiến hóa, bao gồm khởi tạo quần thể, chọn lọc, elitism và điều kiện dừng, về bản chất tương tự như GA đã trình bày trong các mục trước; khác biệt duy nhất nằm ở đối tượng thao tác là **cây** thay vì **vector** số hoặc hoán vị. Chọn lọc giải đấu với kích thước giải đấu 3–7 được ưa chuộng trong GP vì đơn giản, ít nhạy với thang đo của hàm thích nghi và dễ song song hóa khi kích thước quần thể lớn. Điều kiện dừng thường là số thế hệ tối đa kết hợp với ngưỡng sai số chấp nhận được; một số hệ thống còn theo dõi độ đa dạng của quần thể để kết thúc sớm khi quần thể đã hội tụ.

Ví dụ minh họa

Xét hai cây cha mẹ với tập toán tử $\{+, -, \times\}$. Cha mẹ thứ nhất là cây $(x+1)^2$; do tập toán tử không có phép lũy thừa, cây này được biểu diễn dưới dạng $\times[+[x, 1], +[x, 1]]$.

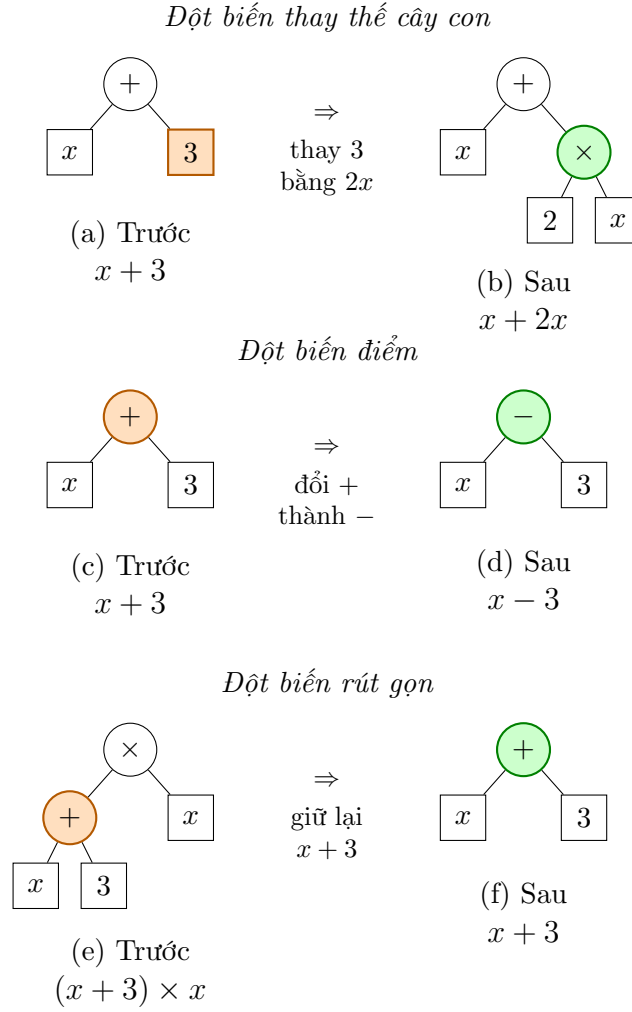
Cha mẹ thứ hai là cây $(x - 1)x$, biểu diễn $\times [-[x, 1], x]$, mang sẵn một *khối xây dựng* (building block) là nhân tử $(x - 1)$.

Hình 1 minh họa phép lai ghép trao đổi cây con giữa hai cha mẹ này. Nút cắt được chọn là nhánh $+[x, 1]$ của cha mẹ thứ nhất và nhánh $-[x, 1]$ của cha mẹ thứ hai – hai nhánh này được tô màu cam trong hình. Hoán đổi hai nhánh cho nhau sinh ra hai cá thể con: $(x - 1)(x + 1) = x^2 - 1$ và $(x + 1)x = x^2 + x$, với phần vừa được ghép sang từ cha mẹ kia tô màu xanh lá. Cả hai cá thể con đều tổ hợp đúng nhân tử tuyến tính của cha mẹ này với phần còn lại của cha mẹ kia, minh họa trực tiếp cách lai ghép cây con kết hợp các khối xây dựng hữu ích từ nhiều cha mẹ – cơ chế tương tự giả thuyết khối xây dựng của GA nhưng vận hành trên cấu trúc phân cấp thay vì trên chuỗi tuyến tính.



Hình 1: Lai ghép trao đổi cây con giữa hai cây cha mẹ (a)–(b), tạo ra hai cây con (c)–(d). Nút cắt tô cam; phần vừa nhận từ cha mẹ kia ở mỗi cây con tô xanh lá.

Ba toán tử đột biến cũng minh họa được trên một cây đơn giản, chẳng hạn cây $x + 3$, như trong Hình 2. Đột biến thay thế cây con chọn một nhánh – ví dụ nhánh 3 – và thay bằng một cây con mới sinh ngẫu nhiên, chẳng hạn $2x$, được cá thể $x + 2x$. Đột biến điểm giữ nguyên cấu trúc cây, chỉ đổi một nút hàm sang hàm khác cùng số ngôi: đổi $+$ thành $-$, được cây $x - 3$. Đột biến rút gọn thao tác trên một cây lớn hơn, chẳng hạn $(x + 3) \times x$: chọn cây con $x + 3$ bên trong nó rồi thay thế toàn bộ cá thể bằng đúng cây con đó, được cá thể mới chỉ còn $x + 3$, nhỏ hơn hẳn cây gốc.

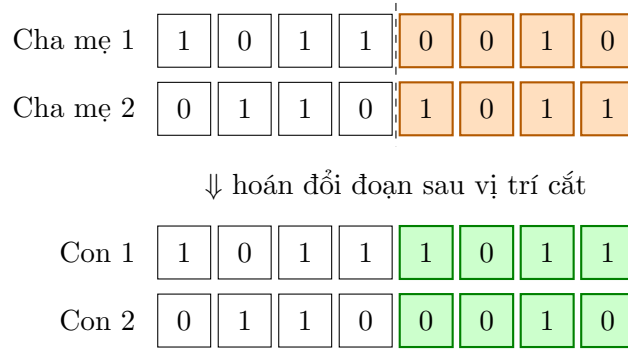


Hình 2: Ba toán tử đột biến minh họa trên cây $x + 3$ (đột biến rút gọn minh họa trên cây lớn hơn $(x + 3) \times x$). Nút hoặc nhánh bị thay đổi tô cam trên cây trước đột biến (a, c, e); nút hoặc nhánh kết quả tô xanh lá trên cây sau đột biến (b, d, f).

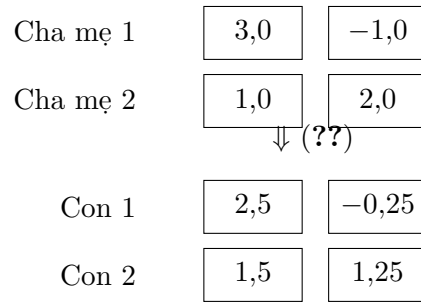
Minh họa bổ sung cho GA (dành cho Mục ?? và Mục ?? của report chính – xem trước ở đây trước khi ghép vào đúng chỗ).

Hình 3 minh họa lai ghép đơn điểm trên mã hóa nhị phân và lai ghép trộn trên mã hóa số thực – hai cách mã hóa được nhắc đến nhiều nhất trong report, lần lượt là cách mã hóa cổ điển của GA và cách mã hóa dùng trong phần cài đặt thực nghiệm của tiểu luận. Với mã hóa nhị phân, vị trí cắt được chọn ngẫu nhiên sau gen thứ 4; đoạn từ vị trí 5 đến 8 (tô cam) được hoán đổi giữa hai cha mẹ để tạo ra hai cá thể con, với đoạn vừa nhận tô xanh lá. Với mã hóa số thực, hai cá thể con được tính trực tiếp theo công thức lai ghép trộn (??) với $\alpha = 0,75$.

Mã hóa nhị phân – lai ghép đơn điểm



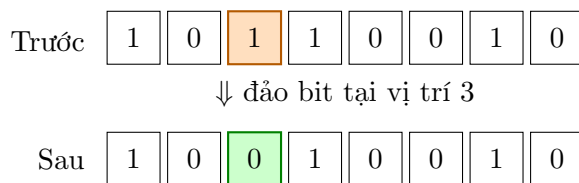
Mã hóa số thực – lai ghép trộn ($\alpha = 0,75$)



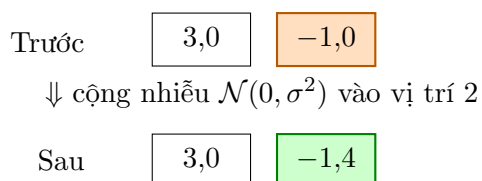
Hình 3: Ví dụ lai ghép trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với lai ghép đơn điểm – đoạn sắp hoán đổi tô cam trên cha mẹ, đoạn vừa nhận được ở cá thể con tô xanh lá. Dưới: mã hóa số thực với lai ghép trộn theo (??), $\alpha = 0,75$.

Tương tự, Hình 4 minh họa đột biến đảo bit trên mã hóa nhị phân và đột biến Gauss trên mã hóa số thực. Với mã hóa nhị phân, gen tại vị trí 3 (tô cam) được đảo giá trị từ 1 thành 0. Với mã hóa số thực, giá trị tại vị trí 2 (tô cam) được cộng thêm một nhiễu Gauss theo (??), ở đây lấy mẫu được $\mathcal{N}(0, \sigma^2) = -0,4$, cho ra giá trị mới $-1,4$ (tô xanh lá).

Mã hóa nhị phân – đột biến đảo bit



Mã hóa số thực – đột biến Gauss



Hình 4: Ví dụ đột biến trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với đột biến đảo bit tại vị trí thứ 3. Dưới: mã hóa số thực với đột biến Gauss, cộng nhiễu ngẫu nhiên vào vị trí thứ 2 theo (??). Vị trí bị thay đổi tô cam trước đột biến; giá trị kết quả tô xanh lá sau đột biến.